

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: ITA0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:100

Annexure A

Experiments

Code of Conduct:

I S. Dharani Teja (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Experiment 1: Naïve Bayes Classifier

Aim

To implement a Naïve Bayes classifier on a real-world dataset, compare different feature sets, evaluate accuracy, precision, and recall, and study the effects of feature independence and Laplace smoothing.

Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, precision_score, recall_score


iris = load_iris()
X = iris.data
y = iris.target


feature_sets = {
    "Petal Features": [2, 3],
    "Sepal Features": [0, 1],
    "All Features": [0, 1, 2, 3]
}


for name, features in feature_sets.items():

    X_train, X_test, y_train, y_test = train_test_split(
        X[:, features], y, test_size=0.2, random_state=42
    )

    model = GaussianNB()
    model.fit(X_train, y_train)
```

```

y_pred = model.predict(X_test)

print("\n", name)
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average="weighted"))
print("Recall  :", recall_score(y_test, y_pred, average="weighted"))

```

Expected Analysis

Feature Set	Expected Performance
Petal features	High
Sepal features	Moderate
All features	Very high

OUTPUT:

```

...
    Petal Features
    Accuracy : 1.0
    Precision: 1.0
    Recall   : 1.0

    Sepal Features
    Accuracy : 0.9
    Precision: 0.9013888888888889
    Recall   : 0.9

    All Features
    Accuracy : 1.0
    Precision: 1.0
    Recall   : 1.0

```

Experiment 2: Maximum Likelihood Estimation vs Bayesian Estimation

Aim

To estimate the **mean and variance** of a probability distribution using **Maximum Likelihood Estimation (MLE)** and **Bayesian estimation**, and study the effect of prior assumptions and sample size on estimation reliability.

Code:

```
import numpy as np

data = np.array([65, 70, 72, 68, 75, 80, 74, 69, 71, 77])

small = data[:4]

large = data[4:]

def mle(data):
    mean = np.mean(data)
    variance = np.mean((data - mean) ** 2)
    return mean, variance

print("Small Sample MLE:", mle(small))
print("Large Sample MLE:", mle(large))

prior_mean = 70
prior_strength = 2

def bayesian_mean(data):
    n = len(data)
    sample_mean = np.mean(data)
    posterior_mean = (
        prior_strength * prior_mean + n * sample_mean
    ) / (prior_strength + n)

    return posterior_mean

print("Small Sample Bayesian Mean:", bayesian_mean(small))
print("Large Sample Bayesian Mean:", bayesian_mean(large))
```

Output:

```
... Small Sample MLE: (np.float64(68.75), np.float64(6.6875))
    Large Sample MLE: (np.float64(72.1), np.float64(18.089999999999996))
    Small Sample Bayesian Mean: 69.16666666666667
    Large Sample Bayesian Mean: 71.75
```

Experiment 3: Bayesian Belief Network

Aim

To construct a **Bayesian Belief Network (BBN)** for a real-world problem, perform inference under different evidence conditions, modify conditional probabilities, and analyze how dependencies affect posterior probabilities.

Code:

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model = DiscreteBayesianNetwork([
    ('Disease', 'Fever'),
    ('Disease', 'Cough')
])

cpd_disease = TabularCPD(
    variable='Disease',
    variable_card=2,
    values=[[0.90], [0.10]]
)

cpd_fever = TabularCPD(
    variable='Fever',
    variable_card=2,
    values=[
        [0.80, 0.20],
        [0.20, 0.80]
    ],
    evidence=['Disease'],
    evidence_card=[2]
```

```

)
cpd_cough = TabularCPD(
    variable='Cough',
    variable_card=2,
    values=[
        [0.70, 0.30],
        [0.30, 0.70]
    ],
    evidence=['Disease'],
    evidence_card=[2]
)

model.add_cpds(cpd_disease, cpd_fever, cpd_cough)
print("Model valid:", model.check_model())

inference = VariableElimination(model)
print("\nNo Evidence:")
print(inference.query(['Disease']))
print("\nEvidence: Fever = Yes")
print(inference.query(
    ['Disease'],
    evidence={'Fever': 1}
))
print("\nEvidence: Fever = Yes, Cough = Yes")
print(inference.query(
    ['Disease'],
    evidence={'Fever': 1, 'Cough': 1}
))

```

Expected Output:

BAYESIAN BELIEF NETWORK

1. No symptoms

$P(\text{Disease}) = 0.1$

2. Evidence: Fever = Yes, Cough = No

$P(\text{Disease} \mid \text{Evidence}) = 0.0688$

3. Evidence: Fever = No, Cough = Yes

$P(\text{Disease} \mid \text{Evidence}) = 0.0256$

4. Evidence: Fever = Yes, Cough = Yes

$P(\text{Disease} \mid \text{Evidence}) = 0.5091$

5. Evidence: Fever = No, Cough = No

$P(\text{Disease} \mid \text{Evidence}) = 0.0143$

MODIFIED PROBABILITY

Changed $P(\text{Fever} \mid \text{Disease})$ from 0.80 to 0.95

New $P(\text{Disease} \mid \text{Fever, Cough}) = 0.6233$

Output:

BAYESIAN BELIEF NETWORK

1. No symptoms

$P(\text{Disease}) = 0.1$

2. Evidence: Fever = Yes, Cough = No

$P(\text{Disease} \mid \text{Evidence}) = 0.16$

3. Evidence: Fever = No, Cough = Yes

$P(\text{Disease} \mid \text{Evidence}) = 0.0609$

4. Evidence: Fever = Yes, Cough = Yes

$P(\text{Disease} \mid \text{Evidence}) = 0.5091$

5. Evidence: Fever = No, Cough = No

$P(\text{Disease} \mid \text{Evidence}) = 0.0118$

MODIFIED PROBABILITY

Changed $P(\text{Fever} \mid \text{Disease})$ from 0.80 to 0.95

New $P(\text{Disease} \mid \text{Fever, Cough}) = 0.5519$

=== Code Execution Successful ===

Experiment 4: Candidate Elimination and Hypothesis Space

Aim

To implement the **Candidate Elimination algorithm** and study how the number of attributes and training examples affects hypothesis-space size, convergence, and generalization.

Code:

```
def candidate_elimination(data, attributes):  
    n = len(attributes)  
    S = ['0'] * n  
    G = [['?'] * n]  
    print("Initial S:", S)  
    print("Initial G:", G)  
    for example, label in data:  
        print("\nExample:", example, "Label:", label)  
        if label == "Yes":  
            G = [g for g in G if all(  
                g[i] == '?' or g[i] == example[i]  
                for i in range(n)  
            )]  
            for i in range(n):  
                if S[i] == '0':  
                    S[i] = example[i]  
                elif S[i] != example[i]:  
                    S[i] = '?'  
        else:  
            if all(  
                S[i] == '?' or S[i] == example[i]  
                for i in range(n)  
            ):  
                print("Negative example encountered.")  
    new_G = []  
    for g in G:  
        if all(  
            g[i] == '?' or g[i] == example[i]  
            for i in range(n)
```



```

    ):
        for i in range(n):
            if g[i] == '?':
                if S[i] != '?' and S[i] != '0':
                    new_g = g.copy()
                    new_g[i] = S[i]
                    if new_g not in new_G:
                        new_G.append(new_g)
            else:
                new_G.append(g)
        G = new_G
        print("S =", S)
        print("G =", G)
    return S, G
attributes = [
    "Sky",
    "AirTemp",
    "Humidity",
    "Wind"
]
data = [
    ("Sunny", "Warm", "Normal", "Strong", "Yes"),
    ("Sunny", "Warm", "High", "Strong", "Yes"),
    ("Rainy", "Cold", "High", "Strong", "No"),
    ("Sunny", "Warm", "High", "Weak", "Yes")
]
S, G = candidate_elimination(data, attributes)
print("\nFinal Specific Hypothesis:")
print(S)

```

```

print("\nFinal General Hypotheses:")
for g in G:
    print(g)
values_per_attribute = 3

for n in range(2, 7):    hypothesis_space = (values_per_attribute + 1) ** n

    print(
        "Attributes:", n,
        "| Hypothesis Space:", hypothesis_space
    )
training_sizes = [1, 2, 4, 8, 16, 32]
initial_space = 4 ** 4
for examples in training_sizes:
    remaining_space = max(
        1,
        initial_space // (2 ** examples)
    )

    print(
        "Training Examples:", examples,
        "| Approx. Remaining Hypotheses:", remaining_space
    )
for n in range(2, 7):
    hypothesis_space = 4 ** n
    mistake_bound = hypothesis_space - 1
    print(
        "Attributes:", n,
        "| Hypothesis Space:", hypothesis_space,
        "| Maximum Mistakes:", mistake_bound
    )

```

)

Output:

```
=====
... CANDIDATE ELIMINATION ALGORITHM
=====
Initial S: ['0', '0', '0', '0']
Initial G: [['?', '?', '?', '?']]

Example: ['Sunny', 'Warm', 'Normal', 'Strong'] Label: Yes
S = ['Sunny', 'Warm', 'Normal', 'Strong']
G = [['?', '?', '?', '?']]

Example: ['Sunny', 'Warm', 'High', 'Strong'] Label: Yes
S = ['Sunny', 'Warm', '?', 'Strong']
G = [['?', '?', '?', '?']]

Example: ['Rainy', 'Cold', 'High', 'Strong'] Label: No
S = ['Sunny', 'Warm', '?', 'Strong']
G = [['Sunny', '?', '?', '?'], ['?', 'Warm', '?', '?'], ['?', '?', '?', 'S

Example: ['Sunny', 'Warm', 'High', 'Weak'] Label: Yes
S = ['Sunny', 'Warm', '?', '?']
G = [['Sunny', '?', '?', '?'], ['?', 'Warm', '?', '?']]

Final Specific Hypothesis:
['Sunny', 'Warm', '?', '?']

Final General Hypotheses:
['Sunny', '?', '?', '?']
['?', 'Warm', '?', '?']

=====
HYPOTHESIS SPACE ANALYSIS
```

Experiment 5: Decision Tree Classifier

Aim

To implement a **Decision Tree classifier** on a real-world dataset using **Entropy** and **Gini Index**, and analyze the effect of tree depth and pruning on accuracy, F1-score, and overfitting.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score
```

```

iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("DECISION TREE CLASSIFIER")
print("=====")
criteria = ["gini", "entropy"]
depths = [2, 3, 5, None]
for criterion in criteria:
    print("\nSplitting Criterion:", criterion)
    for depth in depths:
        model = DecisionTreeClassifier(
            criterion=criterion,
            max_depth=depth,
            random_state=42
        )
        model.fit(X_train, y_train)
        train_pred = model.predict(X_train)
        test_pred = model.predict(X_test)
        train_accuracy = accuracy_score(y_train, train_pred)
        test_accuracy = accuracy_score(y_test, test_pred)

        test_f1 = f1_score(
            y_test,
            test_pred,
            average="weighted"

```

)

```
print("\nDepth:", depth)
print("Training Accuracy:", round(train_accuracy, 4))
print("Testing Accuracy :", round(test_accuracy, 4))
print("Testing F1 Score :", round(test_f1, 4))
```

```
print("\n=====")
print("PRUNING EXPERIMENT")
print("=====")
tree_without_pruning = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
tree_without_pruning.fit(X_train, y_train)
pred1 = tree_without_pruning.predict(X_test)
accuracy1 = accuracy_score(y_test, pred1)
f1_1 = f1_score(y_test, pred1, average="weighted")
tree_with_pruning = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    random_state=42
)
tree_with_pruning.fit(X_train, y_train)
pred2 = tree_with_pruning.predict(X_test)
accuracy2 = accuracy_score(y_test, pred2)
f1_2 = f1_score(y_test, pred2, average="weighted")
print("\nWithout Pruning")
print("Accuracy:", round(accuracy1, 4))
print("F1 Score:", round(f1_1, 4))
print("\nWith Pruning")
```

```
print("Accuracy:", round(accuracy2, 4))  
print("F1 Score:", round(f1_2, 4))
```

Output:

```
...  DECISION TREE CLASSIFIER  
      =====  
  
      Splitting Criterion: gini  
  
      Depth: 2  
      Training Accuracy: 0.95  
      Testing Accuracy : 0.9667  
      Testing F1 Score  : 0.9664  
  
      Depth: 3  
      Training Accuracy: 0.9583  
      Testing Accuracy : 1.0  
      Testing F1 Score  : 1.0  
  
      Depth: 5  
      Training Accuracy: 0.9917  
      Testing Accuracy : 1.0  
      Testing F1 Score  : 1.0  
  
      Depth: None  
      Training Accuracy: 1.0  
      Testing Accuracy : 1.0  
      Testing F1 Score  : 1.0  
  
      Splitting Criterion: entropy  
  
      Depth: 2  
      Training Accuracy: 0.95  
      Testing Accuracy : 0.9667  
      Testing F1 Score  : 0.9664
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand